



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

*Distributed
Computing*



Evaluating Generative Models on Musical Mel Spectrogram Inversion

Distributed Systems Lab

Lucas Erlacher

`elucas@ethz.ch`

Distributed Computing Group
Computer Engineering and Networks Laboratory
ETH Zürich

Supervisors:

Luca Lanzendörfer, Nathanaël Perraudin
Prof. Dr. Roger Wattenhofer

February 28, 2024

Abstract

While spectrograms, particularly log Mel spectrograms, are common representations in generative audio pipelines, converting them to waveforms is challenging due to the non-invertibility of the Mel transformation. Motivated by this we have explored a selection of generative models for the inversion of Mel spectrograms with a focus on the area of music generation. All models were evaluated on an array of metrics and compared to suitable baselines. Surprisingly, the pseudoinverse baseline exhibits strong performance across metrics, though it suffers from audible artifacts. This suggests that it is a good starting point for further refinement by machine learning models. Among the data-driven models, the 2-dimensional CNN is the overall strongest performer (both when comparing the metrics and inspecting the generations by ear). Notably, our implementation of diffusion models, despite their theoretical appeal, do not outperform conventional architectures in our setting, motivating further investigation into diffusion architectures for Mel spectrogram inversion.

Contents

Abstract	i
1 Introduction	1
2 Related Work	2
2.1 Diffusion Models	2
2.2 Speech Vocoder	3
2.2.1 HiFi-GAN	3
2.2.2 DiffWave	3
2.3 Music Vocoder	4
2.3.1 Mel Spectrogram Inversion with Stable Pitch	4
3 Model Architectures	5
3.1 Baselines	5
3.1.1 Pseudoinverse Baseline	5
3.1.2 Pre-Trained Baselines	5
3.2 Implemented Models	6
3.2.1 CNNs	6
3.2.2 U-Net	6
3.2.3 Diffusion Model	7
4 Training & Evaluation	8
4.1 Dataset	8
4.2 Evaluation Metrics	9
4.3 Training Setup	10
4.4 Evaluation Results	10
5 Conclusion and Future Work	13

CONTENTS

iii

Bibliography

14

Introduction

Spectrograms and in particular log Mel spectrograms are a common representation in generative audio pipelines. The Mel scale [1] is a perceptual scale of pitches which approximates the human ear's response to different frequencies of sound waves. Typically, a log Mel spectrogram is the final output of a generative architecture. However, in order to convert a log Mel spectrogram into a waveform (which would typically be done using the inverse short-term Fourier transform (ISTFT)) it first needs to be transformed into a "regular" spectrogram i.e. a spectrogram that is not using the Mel scale. More precisely, we want to transform $S_{log_Mel} = \log(m(S))$ into S where S is the magnitude component of the short-term Fourier transform of a waveform. A system carrying out such a transformation is called a vocoder. Since the Mel transformation is generally non-invertible we must resort to inexact methods. While the problem of log Mel spectrogram inversion has been studied for speech ([2], [3], [4], [5]) it has not received as much attention for musical data. As music poses its unique set of challenges such as sustained notes at a stable pitch and polyphony applying speech vocoders typically does not yield satisfactory results ([6]). We were therefore interested in investigating the performance of a selection of machine learning models on the problem of (musical) Mel spectrogram inversion.

Related Work

2.1 Diffusion Models

Denoising diffusion probabilistic models (DDPMs) - diffusion models for short - have been successfully applied to various domains including text to image ([7], [8]), text to 3D model ([9]) or text to video ([10]). The high level idea of diffusion is to sample an initial tensor of isotropic gaussian noise (this sampling operation ensures that the generations are not deterministic) from which small amounts of noise are then iteratively removed in order to "reveal" the final generation (e.g. an image). This denoising process is typically carried out by a u-net and can e.g. be guided by a text prompt ([7]). The training of the u-net of the diffusion model consists of first randomly sampling a timestep t and then adding the appropriate amount of noise to a ground truth datapoint x_0 using the noise schedule β_t (this process is called forward diffusion). Formally one step in the forward diffusion process is given by ([11]):

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$$

where ϵ is isotropically gaussian. The closed form for deriving x_t from x_0 is given by:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ and ϵ is distributed as before. Finally, the u-net of the diffusion model is trained to predict (given the noisy datapoint x_t as input) the noise ϵ that was used to obtain x_t from x_0 via the closed form above. In the original paper by Ho et al. ([11]) this training is carried out using gradient descent and the loss function $L(y, \hat{y}) = ||y, \hat{y}||^2$. Various additions to diffusion models have been proposed over time such as operating the u-net of the diffusion model on data that has been encoded/decoded into/from a latent code using a variational autoencoder (VAE) ([8]).

We now present a selection of previous work that relates to our project.

2.2 Speech Vcoders

The majority of previous work on log Mel spectrogram inversion has focused on speech. In the following we review a selection of relevant papers from this area.

2.2.1 HiFi-GAN

"HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis" ([2]) aimed to improve on the inference speed of existing flow based models by applying a generative adversarial network (GAN) to the problem of log Mel spectrogram inversion. The proposed architecture consists of a generator and two discriminators. The generator is a fully convolutional architecture that upsamples the input (= a log Mel spectrogram) with transposed convolutions. Every convolution is followed by a so called multi-receptive field fusion module which is designed to simultaneously analyze patterns of differing durations. The choice of employing two discriminators is motivated by the need to capture long term dependencies in order to accurately model speech. To this end, the authors make use of a so-called multi-period discriminator (designed to analyze different periodic patterns in the input) and a multi scale discriminator (designed to analyze the input as a whole which is necessitated by the fact that the first discriminator operates on disjoint chunks of the input). The loss function used to train HiFi-GAN contains (next to typical GAN objectives for generator and discriminator) a Mel spectrogram and a "feature matching" component, the Mel spectrogram component being the L1 distance between the Mel spectrogram derived from the output of the generator and the ground truth transformed into a Mel spectrogram while the feature matching component is measuring the difference between discriminator features of synthetic and target samples.

2.2.2 DiffWave

The authors of "DiffWave: A Versatile Diffusion Model for Audio Synthesis" ([4]) made use of a diffusion model for either unconditional or conditional (using a log Mel spectrogram) audio generation. The neural network of the diffusion model consists of bidirectional dilated convolutional layers that contain residual connections between them. The timestep t was encoded as a 128 dimensional vector which was subsequently fed into a fully connected network before being repeatedly added to the input of the residual layers of the CNN. In case of conditionally generating an output the conditioner (= a Mel spectrogram) was added to the residual layers of the CNN as a bias term (after transforming it to the appropriate size using transposed 2D convolutions). Unlike HiFi-GAN, DiffWave is trained using a single training objective i.e. is not using additional loss components (e.g. a Mel spectrogram term).

2.3 Music Vcoders

Recently, the inversion of Mel spectrograms representing Music has received increased attention, the most notable work being the following paper.

2.3.1 Mel Spectrogram Inversion with Stable Pitch

Noticing the inability of vocoders trained on speech to invert Mel spectrograms representing music the authors of "Mel Spectrogram Inversion with Stable Pitch" ([6]) trained a vocoder specialized on music. The contributions of the paper include a convolutional architecture trained on this task, a phase integration algorithm and an evaluation metric designed to measure pitch stability. The proposed network consists of 8 convolutional layers and takes as it's input a Mel spectrogram. The authors then trained it to predict both a magnitude spectrogram and the two partial derivatives (one along the time axis and one along the frequency axis) of the respective phase spectrogram. The loss function used to train all channels is the MSE with additional components specific to the channel. At inference time the proposed phase integration algorithm (which according to the authors increases computational efficiency and perceived reconstruction quality) is used to estimate the phase spectrogram from the predicted phase gradients. Given the resulting phase spectrogram and the respective magnitude spectrogram a waveform can be constructed (using the ISTFT).

Model Architectures

3.1 Baselines

We have implemented a variety of baselines to compare the performance of our implemented models to. We discuss the implementations of the baselines in the following.

3.1.1 Pseudoinverse Baseline

Our first baseline simply applies a non data-driven transformation to the log Mel spectrogram to produce its output. This transformation approximates the inverse of the steps used to compute the log Mel spectrogram of the dataset. Firstly, the range of the log Mel spectrogram (which is assumed to be in $[0, 1]$) is scaled by the upper bound of the dynamic range of all log Mel spectrograms ($= 50$). The resulting spectrogram is then shifted into the range $[-50, 0]$ (recall that -50 is the value we clipped the negative peaks of the log Mel spectrograms to during dataset computation) before applying the inverse decibel transform to it. Finally, the resulting spectrogram is multiplied by the pseudoinverse of the Mel basis that was used in the computation of the dataset to obtain the output of this baseline.

3.1.2 Pre-Trained Baselines

On top of our trained baselines we have also integrated select pre-trained baselines: HiFi-GAN [2] (more precisely it's "VCTK_V1" checkpoint containing 13.92M parameters) and BigVGAN [3] (more precisely it's "bigvgan_22khz_80band" checkpoint containing 112M parameters). Both are GAN based vocoders trained to invert a Mel spectrogram. HiFi-GAN has been discussed in 2.2.1. BigVGAN tries to improve on previous work by utilizing the Snake activation function ([12]) and training at a larger scale than previously done. Acknowledging that both HiFi-GAN and BigVGAN are trained on speech data we set out to incorporate a

baseline trained on music data. The only work we were able to locate was musicLDM2 which offers a checkpoint trained on music data. However, the vocoder of said checkpoint is a HiFi-GAN variant trained on speech data and unfortunately was not fine-tuned on music data.

3.2 Implemented Models

3.2.1 CNNs

As our first model we have implemented two convolutional neural networks, a 1-dimensional version and a 2-dimensional version. The 1-dimensional CNN is made up of 6 convolutional layers where the first three layers keep doubling the number of input channels and the latter three keep halving the number of input channels. The number of input channels of the first convolutional layer is 513 (number of frequency bins of the STFT divided by 2 plus 1). All layers use a kernel size of 5. The outputs of the first five convolutional layers is fed into a ReLU activation function and subsequently a 1-dimensional batch-norm operation. The output of the 6-th layer is fed into a sigmoid function in order to obtain an output spectrogram in $[0, 1]$. The 2-dimensional CNN is analogous to its 1-dimensional counterpart except for the usage of 2-dimensional convolutions/batch-norm operations and the fact that layer 1 and 6 increase/decrease the number of channels by a factor of 8 (which we have chosen in order to give the 2-dimensional network additional modeling power by increasing its parameter count). Moreover, the number of input channels of the first layer of the 2D CNN is 1 (typically this value would be 3 for images due to the RGB channels of an image but a spectrogram only contains 1 channel). Note that the input to both CNN's is not the first element of a datapoint from the dataset (= a normalized spectrogram) but the pseudoinverse approximation that was described in 3.1.1. Furthermore, both this starting point (before being passed to the CNN's) and the target spectrogram are transformed by a log operation in an effort to pronounce low volume details in the spectrograms. Both CNN's were trained to predict the difference between their input and the target spectrogram. The 1D CNN ended up having a parameter count of 110M and its 2D counterpart a count of 32.6K (this size discrepancy is mainly caused by the different use of channels in the two implementations).

3.2.2 U-Net

As our next model we chose to use the pre-existing Unet1d implementation by Huggingface from their diffusers library (we omit the implementations details of this class as they are not part of our work). The reason for choosing this specific implementation was to be as comparable as possible to the diffusion model (which internally makes use of the same U-Net). The Unet1d class is constructed with

sample_size being the length of the input spectrogram divided by the hop size of the STFT and in_channels and out_channels being 513 (number of frequency bins of the STFT divided by 2 plus 1). This way the convolutions of the U-Net (which are 1-dimensional) end up processing the input spectrogram analogously to our 1-dimensional CNN. In order to disable any diffusion related functionality from the U-Net we sample a constant vector of timesteps filled with zeros. Afterwards, the forward method of the UNet1DModel class is invoked whose output is fed into a sigmoid function (in order to obtain a spectrogram in the range $[0, 1]$) before being returned. As with the convolution based models the U-Net takes as input the pseudoinverse approximation and both the input to the network and the target spectrogram are transformed by a log operation. Furthermore, the U-Net was trained to predict the difference between it's input and the target. The U-Net ended up having a parameter count of 2.8M.

3.2.3 Diffusion Model

As a starting point for the implementation of our diffusion model we have made use of the tqdne library (<https://github.com/highfem/tqdne>). This library is based on Huggingface's diffusers library and provides code for training a diffusion based upscaler. We have adapted to library code to during training concatenate the noisy target spectrogram (i.e. a target spectrogram from the dataset that has been perturbed by noise according to the timestep t and the noise schedule) with the pseudoinverse approximation. This is what is then passed to the U-Net of the diffusion model as input. Unlike diffusion was originally described in ([11]) we don't train the U-Net to predict the noise but to directly predict the noise-free spectrogram from the dataset (this is called "sample mode" in tqdne). Note that as with all non-pretrained baselines the target spectrogram and the pseudoinverse approximation were transformed by a log operation. At inference time we perform the same concatenation of noise and pseudoinverse approximation, invoke the diffusion pipeline (which iteratively applies the U-Net of the diffusion model) and clip the output at 0 and 1 to obtain an output in the desired range. Note that at inference time we do in fact invoke the U-Net multiple times (and not just once as one might believe given the sample-mode training procedure). We have trained and evaluated 3 different diffusion model versions: one with 10 inference steps, one with 100 and one with 1000 inference steps. All diffusion models ended up having a parameter count of 2.9M.

Training & Evaluation

4.1 Dataset

For training and evaluation we have used a dataset made up of music samples. The audio clips of our dataset are from the Maestro v 3.0.0 dataset ([13]). It is a well known collection of high quality piano recordings of classical music (it also contains MIDI files of the recordings but we did not make use of them in this project). The train/test split was made using the folder structure of this dataset where the folders "2004" to "2014" were used for training and the folders "2015" to "2018" were used for testing. While we acknowledge that this way of creating the split may introduce some systematic bias into the train and test splits (e.g. maybe the folders corresponding to later years contain a slightly different style of music than the folders corresponding to earlier years) it constituted the simplest way of implementing the train test split given the size of the Maestro dataset (which we appreciated given the time constraints of this project). To obtain our dataset, the Maestro recordings were loaded into memory and split into pieces of 32768 samples for the training dataset and 65536 samples for the evaluation dataset (the smaller number of samples in training was necessitated by the memory size of the GPU we trained on). The audio pieces were then transformed into normalized spectrograms (omitting pieces that were shorter than the piece length of the training/evaluation datasets to avoid spectrograms of differing length in the time dimension) where the spectrogram was obtained via STFT (using a hop size of 256 and 1024 frequency channels) and the norm operation transformed the input into the range $[0, 1]$. These normalized spectrograms made up the ground truth (i.e. the second element of a datapoint from the dataset). To obtain the input to the networks (i.e. the first element of a datapoint from the dataset) the normalized spectrogram was transformed into a log Mel spectrogram by matrix-multiplying it with the Mel basis (using 80 Mel channels), transforming the result to the decibel scale, clipping highly negative values to -50 and finally moving the range of the spectrogram to $[0, 1]$. Note that in the interest of keeping the runtime of the evaluation reasonable we had to limit the amount of evaluation samples to 1000.

4.2 Evaluation Metrics

A natural metric to measure the quality of model outputs was the MSE (between model output and ground truth spectrogram) since it was the loss function used to train our models. Computing the L1 distance to the target spectrogram was a good complementary choice as it is less outlier sensitive than the MSE.

Cosine-based metrics, in addition to L1 distance, provided the added benefit of not penalizing "systematic" volume differences. For instance, imagine the model produces a spectrogram identical to the target, differing only in a subtle -2dB volume shift. In such cases, we would want our metrics to highlight the similarity between the two (despite the volume variance), a task neither MSE nor L1 would accomplish due to their inability to account for volume differences. To obtain such "volume invariant" distance measurements we computed a row-wise (i.e. compute the cosine distance for each pair of output-target rows and then average those similarities), column-wise (same as before but with pairs of columns) and flattened (i.e. flatten both spectrograms into 1D vectors and compute the cosine distance between those) cosine distance. Another metrics regularly used in related works is the Frechet Audio Distance (FAD) ([14]). It is designed to measure how a provided audio clip compares to clean, studio-recorded music (this is accomplished by comparing embeddings extracted from the input clip with those from a database of background music) which seemed suitable for our context as the model outputs often contained a certain amount of noise and pitch instability.

Our final evaluation metric was motivated by the fact that typically the final "product" of a generative audio pipeline is not a spectrogram but an audio clip that a user can listen to. However, since the conversion from spectrogram to waveform (typically done via the ISTFT) can introduce perceptual artifacts (e.g. distortion) which are not easily measurable when comparing spectrograms we wanted to create a distance measurement between waveforms rather than spectrograms. To this end we devised two metrics: a baseline measurement which quantifies the amount of deviation introduced by the ISTFT alone and the desired distance on waveforms. The baseline measurement is computed by taking $\text{MSE}(\text{STFT}(\text{ISTFT}(y)), y)$ where the MSE is computed over the evaluation set. This way we obtain a measurement of the amount of perturbation introduced into the evaluation set by simply applying the ISTFT (which can be used to put the value of the waveform distance into perspective). The distance on waveforms is computed by taking $\text{MSE}(\text{STFT}(\text{ISTFT}(\hat{y})), y)$ where the MSE is computed over the entire evaluation set and $\hat{y}$ denotes the model output with y being the associated ground truth spectrogram. Note that this distance (while not directly being computed in waveform space but instead on spectrograms) does indeed meet our goal of "quantifying what a listener would hear" because we apply the ISTFT operation to the network output thereby simulating the introduction of ISTFT artifacts that would occur if the model output (a spectrogram) would be presented to a listener.

4.3 Training Setup

All our models were trained with gradient descent using the Adam optimizer with a learning rate of 0.0001 and a batch size of 32. All non-diffusion models were trained for 24 hours. The diffusion models with 10 and 100 inference steps were trained for 24 hours and the diffusion model with 1000 steps was trained for 48 hours. All models were trained on a NVIDIA TITAN XP with 12 GB of RAM.

4.4 Evaluation Results

The results of evaluating all baselines and trained models on using the above described metrics are summarized in the following table:

	MSE(10^{-2})	L1(10^{-6})	cos _{row}	cos _{col}	cos _{flat}	F	W(10^{-3})
pseudoinv	0.01	2.18	0.97	0.91	0.975	20.0	0.12
Hifi-GAN	0.04	9.76	0.89	0.87	0.88	23.3	0.4
BigVGAN	0.009	4.79	0.96	0.90	0.978	6.2	0.1
CNN 1D	0.74	3.62	0.91	0.91	0.90	4.3	0.5
CNN 2D	0.62	2.49	0.98	0.93	0.97	3.5	0.3
U-Net	0.77	3.01	0.94	0.92	0.93	6.9	0.3
diff_10	1.15	2.86	0.92	0.90	0.91	4.9	0.3
diff_100	0.81	2.94	0.92	0.91	0.91	8.6	0.3
diff_1000	0.94	2.97	0.92	0.90	0.90	5.2	0.4

Table 1: evaluation results of all baselines and models across different metrics.

Value of best scoring model with respect to each metric is marked in bold.

Note that "F" denotes the Frechet Audio Distance, "W" denotes the waveform distance discussed above and that the value of the waveform distance baseline was 1.2420e-05. "pseudoinv" denotes the pseudoinverse approximation of the Mel transform and "diff_n" denotes the diffusion model with n inference steps. Values within the same metric can be compared as follows: MSE - lower is better, L1 - lower is better, cosine distances - higher is better, FAD - lower is better, and waveform distance - lower is better.

Inspecting the table of results a few findings can be made. Firstly, Hifi-GAN performs comparatively poorly on metrics such as the cosine metrics and the FAD. This is not surprising as it was not trained on music data. BigVGAN performs noticeably better across all metrics (even being the highest scoring model in 3 categories) likely due to the increased size and training time.

Surprisingly, the pseudoinverse baseline performs very well (if not taking the first place) across multiple metrics such as MSE, L1, cosine metrics and waveform distance. This is surprising, as the outputs of this model in general contain noticeably more audible artifacts (some of which are described in the last paragraph of this section) than the generations of e.g. the 2 dimensional CNN. This discrepancy in audio quality is however reflected in certain metrics (e.g. the FAD of the 2-dimensional CNN is significantly lower than the score of the pseudoinverse baseline). Taken together these results are certainly surprising and might raise questions about the meaningfulness of the non-FAD metrics in our setting. The observed results do however validate our choice of using the pseudoinverse spectrogram as the starting point for the refinement with our implemented neural networks.

The 2-dimensional CNN is the best performing model in certain metrics (column wise cosine similarity, FAD) which matches the perceived generation quality as this model seemed to produce the "cleanest" output. The 1-dimensional CNN performs worse than its 2-dimensional counterpart which is not particularly surprising considering it can only operate on the input with 1-dimensional kernels.

Another rather surprising finding of the evaluation was that our implementation of diffusion was generally outperformed by the 2-dimensional CNN. This is surprising as diffusion has proven to be superior to convolutional architectures in other domains such as image generation. As mentioned in the last chapter it is very much possible that architectural adaptations to the diffusion model are necessary to unlock its full capabilities in our setting. The discrepancy between 2-dimensional CNN and diffusion models is however consistent with the results of the qualitative evaluation of model outputs (see last paragraph of section). Furthermore (disregarding FAD) the different versions of the diffusion model do not differ significantly from each other and no clear improvement/trend can be observed when increasing the number of inference steps. Finally, our U-Net implementation is not clearly superior or inferior to our diffusion implementations suggesting that the potential of diffusion on Mel spectrogram inversion is not yet fully achieved.

Evaluating the generations of the trained models more qualitatively we noticed that all trained models were most of the time able to significantly reduce the characteristic "metallic buzz" of the pseudoinverse spectrograms. Moreover, all models were most of the time able to reduce the at times present slight pitch instabilities of the pseudoinverse spectrograms and outputs of the speech models (in particular Hifi-GAN). An issue that we were able to observe is that the U-Net and diffusion architectures struggle to output the characteristic attack of playing piano keys. In the outputs of these models the beginning of a note sounds very soft and "smeared". Given that the 2 dimensional CNN does not struggle with this inclusion of the necessary transients while the 1-dimensional CNN does at times the use of 2-dimensional convolutions might be one way to improve this

aspect of the outputs of the U-Net/diffusion model.

In an effort to emulate a user study (which due to time constraints could not be conducted in full) we listened to 5 generations of each model (using the same inputs each time), rated the quality output on a scale of 1-5 (the reviewer did not know which model a sample originated from) and computed the average as the Mean Opinion Score (MOS) score of each model. The obtained values are largely consistent with the previous evaluation. The scores are summarized in the following table:

	MOS
pseudoinv	2.4
Hifi-GAN	1.0
BigVGAN	3.0
CNN 1D	2.0
CNN 2D	4.0
U-Net	2.0
diff_10	2.4
diff_100	2.2
diff_1000	2.6

Abbreviations of model names are consistent with table 1. Note that a higher MOS score indicates better quality. The highest scoring model's value is marked in bold.

Conclusion and Future Work

In conclusion, we have experimented with multiple different models on the problem of Mel spectrogram inversion. All models were evaluated on an array of metrics and compared to appropriate baselines. The results of the evaluation indicate that using the described pseudoinverse is a suitable starting point which can afterwards be improved by certain models such as the 2-dimensional CNN.

We do however notice that the following limitations/challenges remain. Firstly, our training data exclusively consists of classical piano recordings. Since this does not represent the entire spectrum of musical instruments and genres further work is needed to investigate the performance of the presented models on other types of recordings. Secondly, due to memory restrictions we had to train our models on relatively short pieces of audio data. Given that audio recordings typically span at least a few minutes further work on merging multiple segments of model output into one coherent multi-minute spectrogram is needed. Furthermore, due to time constraints we had to limit ourselves to the pre-defined U-Net/diffusion architectures offered by the used libraries (diffusers, tqdne) i.e. were not able to experiment with different model architectures/hyperparameters. Considering our evaluation results, the use of 2-dimensional convolutions seems like a simple yet promising next step.

Bibliography

- [1] J. Volkmann, “A Scale for the Measurement of the Psychological Magnitude Pitch,” *Acoustical Society of America Journal*, vol. 8, no. 3, p. 208, Jan. 1937.
- [2] J. Kong, J. Kim, and J. Bae, “Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis,” 2020.
- [3] S. gil Lee, W. Ping, B. Ginsburg, B. Catanzaro, and S. Yoon, “Bigvgan: A universal neural vocoder with large-scale training,” 2023.
- [4] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, “Diffwave: A versatile diffusion model for audio synthesis,” 2021.
- [5] D. Yang, J. Yu, H. Wang, W. Wang, C. Weng, Y. Zou, and D. Yu, “Diffsound: Discrete diffusion model for text-to-sound generation,” 2023.
- [6] B. D. Giorgi, M. Levy, and R. Sharp, “Mel spectrogram inversion with stable pitch,” 2022.
- [7] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, “Hierarchical text-conditional image generation with clip latents,” 2022.
- [8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” 2022.
- [9] C.-H. Lin, J. Gao, L. Tang, T. Takikawa, X. Zeng, X. Huang, K. Kreis, S. Fidler, M.-Y. Liu, and T.-Y. Lin, “Magic3d: High-resolution text-to-3d content creation,” 2023.
- [10] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts, V. Jampani, and R. Rombach, “Stable video diffusion: Scaling latent video diffusion models to large datasets,” 2023.
- [11] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” 2020.
- [12] L. Ziyin, T. Hartwig, and M. Ueda, “Neural networks fail to learn periodic functions and how to fix it,” 2020.

- [13] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, “Enabling factorized piano music modeling and generation with the maestro dataset,” 2019.
- [14] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, “Fréchet audio distance: A metric for evaluating music enhancement algorithms,” 2019.